

INSTITUTO FEDERAL DE MINAS GERAIS (IFMG)
CAMPUS BAMBUÍ
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

Disciplina: BiSuEEA.512 – Sistemas Embarcados
Professor: Williams L. Nicomedes

Henrique Augusto G. Fernandes

TRABALHO Nº1 – CONTADOR MOD 8

SUMÁRIO

1	INTRODUÇÃO	3
2	DESENVOLVIMENTO	4
2.1	Fundamentos Teóricos	4
2.1.1	<i>Circuitos Sequenciais vs. Combinacionais</i>	4
2.1.2	<i>Flip-Flop tipo JK</i>	4
2.1.3	<i>Princípio de Funcionamento do Contador MOD 8</i>	4
2.2	Implementação do Flip-Flop JK em VHDL	5
2.2.1	<i>Estrutura do Código</i>	5
2.2.2	<i>Justificativas Técnicas</i>	5
2.3	Construção do Esquemático do Contador MOD 8	7
2.3.1	<i>Geração do Bloco Lógico</i>	7
2.3.2	<i>Diagrama Esquemático</i>	7
2.3.3	<i>Descrição das Conexões</i>	8
2.3.4	<i>Configuração do Projeto</i>	8
2.4	Simulação e Validação	8
2.4.1	<i>Configuração das Formas de Onda</i>	8
2.4.2	<i>Resultados da Simulação</i>	9
2.4.3	<i>Análise dos Resultados</i>	9
2.4.4	<i>Diagrama de Tempo Teórico</i>	10
3	CONCLUSÃO	11

1 INTRODUÇÃO

Este documento apresenta a resolução do Trabalho nº1 da disciplina de Sistemas Embarcados, referente ao 2º semestre de 2025. O objetivo consiste em projetar e implementar um contador MOD 8 (módulo 8) utilizando flip-flops tipo JK descritos em linguagem VHDL.

Conforme apresentado nas Notas de Aula 20, os circuitos sequenciais diferem dos combinacionais por possuírem elementos de memória, onde a saída depende não apenas das entradas atuais, mas também do estado anterior do circuito. O flip-flop tipo JK é um elemento fundamental para a construção de contadores, registradores e máquinas de estados.

O trabalho está estruturado nas seguintes etapas:

- a) entendimento do funcionamento do contador MOD 8;
- b) implementação do flip-flop tipo JK em VHDL;
- c) construção do esquemático do contador a partir dos flip-flops;
- d) verificação do funcionamento através de simulação.

O desenvolvimento foi realizado no software Intel Quartus II, utilizando a técnica de **processos** para implementação do flip-flop JK e interconexão de múltiplas instâncias para formar o contador.

2 DESENVOLVIMENTO

2.1 Fundamentos Teóricos

2.1.1 Circuitos Sequenciais vs. Combinacionais

A distinção fundamental entre os dois tipos de circuitos digitais está apresentada na Tabela 1.

Tabela 1 – Comparação entre circuitos combinacionais e sequenciais

Característica	Combinacional	Sequencial
Dependência	Apenas entradas atuais	Entradas + estado anterior
Memória	Não possui	Possui (flip-flops)
Atualização	Instantânea	Na transição do clock
Exemplos	Decodificadores, MUX	Contadores, registradores

Fonte: Elaborado pelo autor, 2026.

2.1.2 Flip-Flop tipo JK

O flip-flop JK é um elemento de memória versátil que implementa quatro operações distintas, conforme sua tabela de transição apresentada na Tabela 2.

Tabela 2 – Tabela de transição do flip-flop tipo JK

J	K	$Q(n+1)$	Operação
0	0	$Q(n)$	Mantém o estado atual
0	1	0	Reset (força saída para 0)
1	0	1	Set (força saída para 1)
1	1	$\overline{Q(n)}$	Toggle (inverte o estado)

Fonte: Elaborado pelo autor, 2026.

Onde:

- $Q(n)$: estado atual da saída;
- $Q(n+1)$: próximo estado da saída (após a transição do clock);
- $\overline{Q(n)}$: complemento do estado atual.

2.1.3 Princípio de Funcionamento do Contador MOD 8

O contador MOD 8 é construído com **3 flip-flops JK** conectados em cascata, permitindo contar de 0 a 7 (8 estados distintos, pois $2^3 = 8$). A sequência de contagem é:

$$000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 100 \rightarrow 101 \rightarrow 110 \rightarrow 111 \rightarrow 000 \quad (2.1)$$

O funcionamento em cascata opera da seguinte forma:

- **FF0** (LSB): recebe o clock principal; com $J=K=1$, alterna a cada pulso;
- **FF1**: recebe a saída Q0 como clock; alterna quando Q0 faz transição;
- **FF2** (MSB): recebe a saída Q1 como clock; alterna quando Q1 faz transição.

Esta configuração implementa um **divisor de frequência**:

- Q0: $f_{Q0} = f_{clock}/2$
- Q1: $f_{Q1} = f_{clock}/4$
- Q2: $f_{Q2} = f_{clock}/8$

2.2 Implementação do Flip-Flop JK em VHDL

O Quadro 1 apresenta o código VHDL desenvolvido para o flip-flop tipo JK, seguindo a metodologia das Notas de Aula 20.

2.2.1 Estrutura do Código

O código VHDL está organizado conforme a estrutura padrão:

- a) **Bibliotecas**: inclusão da biblioteca `ieee` e do pacote `std_logic_1164`, necessário para utilizar a função `falling_edge()`;
- b) **Entidade** (`entity`): declaração das portas:
 - J, K: entradas de controle do tipo `std_logic`;
 - `clk`: sinal de clock do tipo `std_logic`;
 - Q: saída do flip-flop;
- c) **Arquitetura** (`architecture`): implementação utilizando:
 - sinal auxiliar `Qstate` para armazenar o estado interno;
 - processo com detecção de borda de descida (`falling_edge`);
 - estrutura `if-elsif` para implementar a tabela de transição.

2.2.2 Justificativas Técnicas

- a) **Sinal auxiliar** `Qstate`: em VHDL, uma porta declarada como `out` não pode ser lida internamente. Como a operação Toggle requer ler o valor atual para invertê-lo, utiliza-se um sinal interno;

Quadro 1 – Código VHDL do flip-flop tipo JK

```

-- =====
-- Flip-Flop tipo JK
-- Disciplina: BiSuEEA.512 - Sistemas Embarcados
-- Descrição: Elemento de memória com transição na borda de
--             descida do clock. Implementa as operações:
--             Manter, Reset, Set e Toggle.
-- =====

library ieee;
use ieee.std_logic_1164.all;

-- Entradas e saídas do Flip-Flop JK
entity BlocoFF_JK is
port(
  J   : in  std_logic;  -- Entrada J
  K   : in  std_logic;  -- Entrada K
  clk : in  std_logic;  -- Sinal de clock
  Q   : out std_logic   -- Saída Q
);
end BlocoFF_JK;

-- Lógica do Flip-Flop
architecture funcionamento of BlocoFF_JK is
-- Sinal auxiliar para armazenar o estado atual
-- Necessário pois não se pode ler uma saída (out)
signal Qstate : std_logic := '0';
begin
-- Processo sensível às entradas J, K e clock
process(J, K, clk)
begin
-- Transição na borda de DESCIDA do clock
if falling_edge(clk) then
-- J=1 e K=1: Toggle (inverte o estado)
if J = '1' and K = '1' then
Qstate <= not(Qstate);
-- J=1 e K=0: Set (força saída para '1')
elsif J = '1' and K = '0' then
Qstate <= '1';
-- J=0 e K=1: Reset (força saída para '0')
elsif J = '0' and K = '1' then
Qstate <= '0';
-- J=0 e K=0: Mantém (implícito)
end if;
end if;
end process;
-- Atribui o estado interno à saída
Q <= Qstate;
end funcionamento;

```

Fonte: Elaborado pelo autor, 2026.

- b) **Inicialização** := '0': garante um estado inicial conhecido durante a simulação;
- c) **Função** falling_edge(clk): detecta a transição de '1' para '0' no clock, conforme especificado nas notas de aula;

- d) **Caso $J=0, K=0$ não implementado**: quando nenhuma condição é verdadeira, Q_{state} mantém seu valor automaticamente, implementando a operação "Manter" de forma implícita.

2.3 Construção do Esquemático do Contador MOD 8

2.3.1 Geração do Bloco Lógico

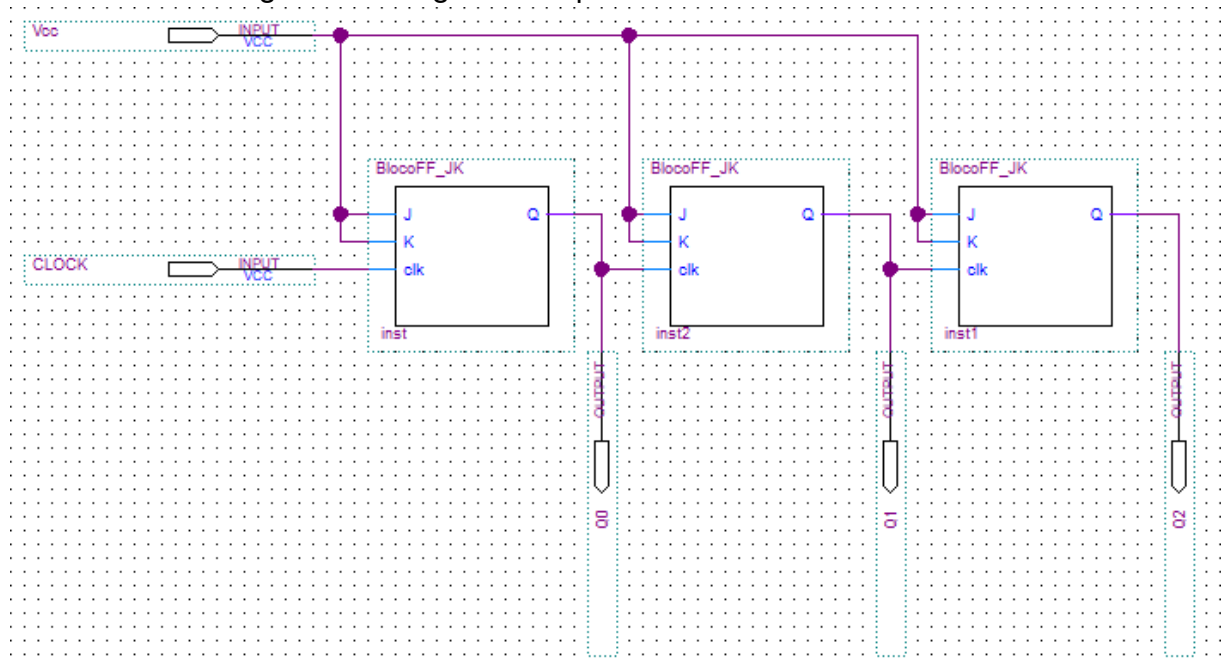
Após a criação do arquivo VHDL, foram executados os seguintes procedimentos:

- Salvar o código como `BlocoFF_JK.vhd`;
- Processing** → **Analyze Current File**;
- File** → **Create/Update** → **Create Symbol Files for Current File**.

2.3.2 Diagrama Esquemático

A Figura 1 apresenta o diagrama esquemático do contador MOD 8, construído no Quartus II com três instâncias do bloco `BlocoFF_JK` interconectadas.

Figura 1 – Diagrama esquemático do contador MOD 8



Fonte: Elaborado pelo autor, 2026.

2.3.3 Descrição das Conexões

Conforme observado na Figura 1, as conexões foram realizadas da seguinte forma:

- a) **Entrada Vcc:** conectada às entradas J e K de todos os flip-flops, mantendo-os em modo Toggle ($J=K=1$);
- b) **Entrada CLOCK:** conectada à entrada `clk` do primeiro flip-flop (inst);
- c) **Cascadeamento:**
 - saída Q de inst $\rightarrow$ entrada clk de inst2;
 - saída Q de inst2 $\rightarrow$ entrada clk de inst1;
- d) **Saídas:**
 - Q0: saída do primeiro FF (inst) – bit menos significativo;
 - Q1: saída do segundo FF (inst2);
 - Q2: saída do terceiro FF (inst1) – bit mais significativo.

2.3.4 Configuração do Projeto

Para compilação correta, o arquivo esquemático foi definido como entidade principal:

- a) No painel **Project Navigator**, clicar com botão direito sobre o arquivo `.bdf`;
- b) Selecionar **Set as Top-Level Entity**;
- c) Executar **Processing** $\rightarrow$ **Start Compilation**.

2.4 Simulação e Validação

2.4.1 Configuração das Formas de Onda

Para validar o funcionamento do contador, foi criado um arquivo de simulação (*University Program VWF*) com as seguintes configurações:

- **End Time:** 1,6 μ s;
- **Clock:** período de 100 ns.

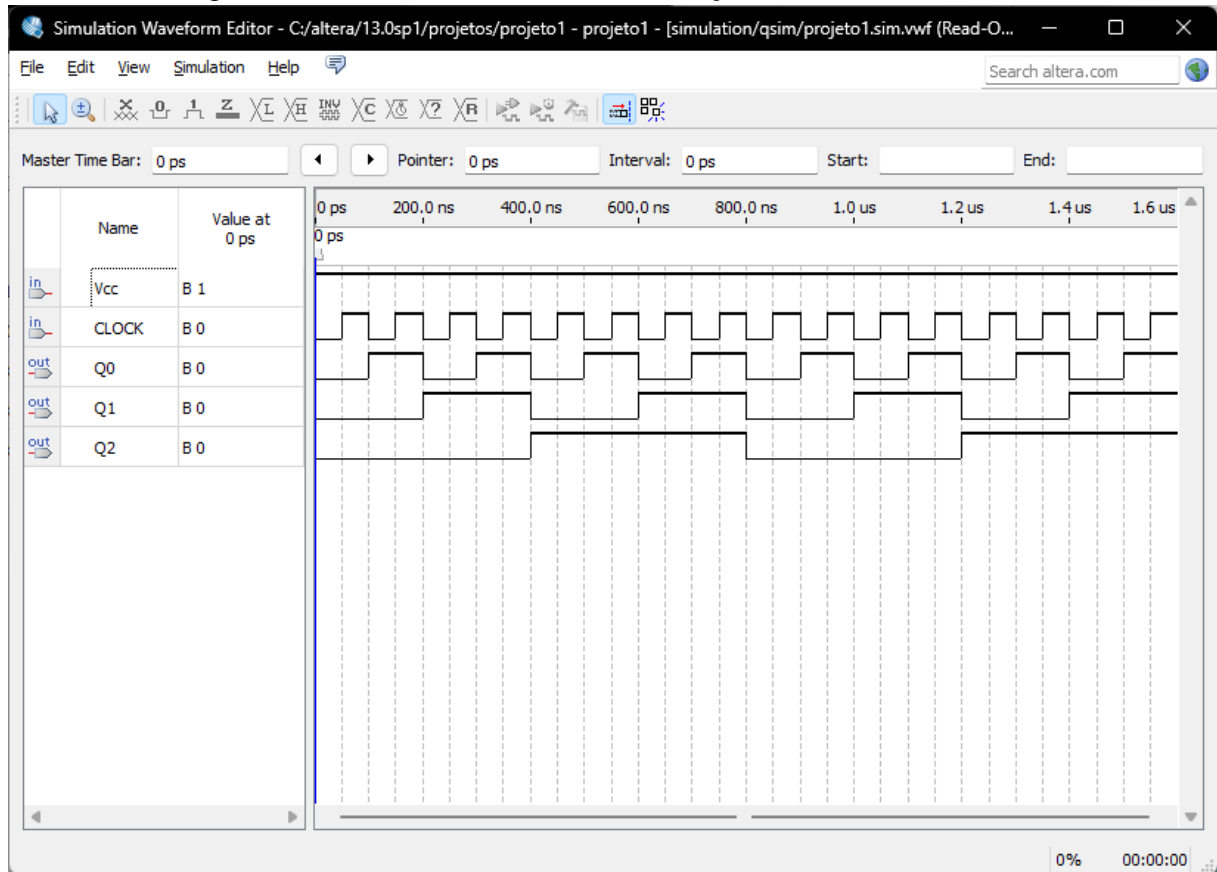
Os sinais configurados foram:

- a) **Vcc:** nível constante '1' (mantém $J=K=1$ em todos os FFs);
- b) **CLOCK:** onda quadrada com período de 100 ns;
- c) **Q0, Q1, Q2:** saídas do contador (observadas).

2.4.2 Resultados da Simulação

A Figura 2 apresenta as formas de onda resultantes da simulação funcional.

Figura 2 – Formas de onda da simulação do contador MOD 8



Fonte: Elaborado pelo autor, 2026.

2.4.3 Análise dos Resultados

Analisando as formas de onda apresentadas na Figura 2, verifica-se o comportamento esperado do contador:

- Q0 (LSB):** alterna a cada borda de descida do clock, com frequência $f_{clock}/2$;
- Q1:** alterna a cada duas bordas de descida do clock principal, com frequência $f_{clock}/4$;
- Q2 (MSB):** alterna a cada quatro bordas de descida do clock principal, com frequência $f_{clock}/8$.

A Tabela 3 apresenta a correspondência entre os pulsos de clock e os estados do contador.

Tabela 3 – Verificação dos estados do contador MOD 8

Pulso	Q2	Q1	Q0	Decimal
0 (inicial)	0	0	0	0
1	0	0	1	1
2	0	1	0	2
3	0	1	1	3
4	1	0	0	4
5	1	0	1	5
6	1	1	0	6
7	1	1	1	7
8	0	0	0	0 (reinicia)

Fonte: Elaborado pelo autor, 2026.

2.4.4 Diagrama de Tempo Teórico

O comportamento temporal esperado do contador pode ser representado da seguinte forma:

- Após 8 pulsos de clock, o contador retorna ao estado inicial (000);
- Cada flip-flop divide a frequência de entrada por 2;
- O ciclo completo de contagem requer 8 períodos do clock principal.

Os resultados obtidos na simulação correspondem integralmente ao comportamento teórico esperado, confirmando a corretude da implementação.

3 CONCLUSÃO

Este trabalho apresentou o projeto e implementação de um contador MOD 8 utilizando flip-flops tipo JK descritos em linguagem VHDL. O desenvolvimento seguiu a metodologia apresentada nas Notas de Aula 20 da disciplina, abordando:

- a) **Fundamentos teóricos:** compreensão da diferença entre circuitos combinacionais e sequenciais, funcionamento do flip-flop JK e princípio de operação do contador em cascata;
- b) **Implementação em VHDL:** desenvolvimento do código do flip-flop JK utilizando processos, detecção de borda de descida e sinal auxiliar para gerenciamento do estado interno;
- c) **Construção do esquemático:** interconexão de três instâncias do bloco flip-flop para formar o contador MOD 8, com cascadeamento das saídas para as entradas de clock;
- d) **Validação por simulação:** verificação completa do funcionamento através de formas de onda no Quartus II.

A simulação funcional validou completamente o comportamento do contador, demonstrando:

- contagem correta de 0 a 7 em sequência binária crescente;
- retorno automático ao estado inicial após o estado 7;
- divisão de frequência correta em cada estágio ($f/2$, $f/4$, $f/8$).